Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Криптография и сетевая безопасность»
Тема: «Избыточное кодирование данных в информационных системах. Код Хемминга и итеративные коды»

Выполнила:
Студентка 4 курса
Группы АС-66
Прокурат В.Д.
Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: приобретение практических навыков кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.

Вариант 9 (4)

Задачи:

1. Закрепить теоретические знания по использованию методов помехоустойчивого кодирования и итеративных кодов для повышения надёжности передачи и хранения в памяти компьютера двоичных данных.
2. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 и 4; итеративным кодом с различной относительной избыточностью кодовых слов.
3. Результаты выполнения лабораторной работы оформить в виде описания разработанного приложения, методики выполнения экспериментов с использованием приложения и результатов эксперимента.

Hamming.py:

```
import argparse
import random

MIN_BITS = 16
ENCODING = "cp1251"


def fmt(bits):
    return "".join(map(str, bits))


def matrix_str(M):
    return "\n".join(" ".join(map(str, row)) for row in M)


def bits_from_text(text):
    data = text.encode(ENCODING, errors="replace")
    return [int(b) for byte in data for b in f"{byte:08b}"]


def calc_r(k):
    r = 2
    while 2 ** r < k + r + 1:
        r += 1
    return r


def build_P(k, r):
    cols, num = [], 3
    while len(cols) < k:
        if num >= 2 ** r:
            raise ValueError("слишком мало проверочных разрядов r для данного k")
        if bin(num).count("1") >= 2:
            cols.append([int(c) for c in format(num, f"0{r}b")])
        num += 1
    return [[cols[j][i] for j in range(k)] for i in range(r)]


def build_H(P, k, r):
    return [P[i] + [1 if j == i else 0 for j in range(r)] for i in range(r)]
```

```
def build_H4(H):
    r, n = len(H), len(H[0])
    Hx = [row + [0] for row in H] + [[1] * (n + 1)]
    last = [sum(Hx[i][j] for i in range(r + 1)) % 2 for j in range(n + 1)]
    Hc = Hx[:-1] + [last]
    return Hx, Hc


def columns(H):
    return [[H[i][j] for i in range(len(H))] for j in range(len(H[0]))]


def check_bits(xk, P, dmin):
    xr = [sum(P[i][j] & xk[j] for j in range(len(xk))) % 2 for i in range(len(P))]
    if dmin == 4:
        xr.append((sum(xk) + sum(xr)) % 2)
    return xr


def syndrome(yn, k, P, dmin):
    r = len(P)
    yk, yr = yn[:k], yn[k:]
    yr2 = check_bits(yk, P, dmin)
    s = [a ^ b for a, b in zip(yr[:r], yr2[:r])]
    if dmin == 4:
        s.append(sum(yn) % 2)
    return yr2, s


def locate(s, P, k, dmin):
    r = len(P)
    n = k + r + (1 if dmin == 4 else 0)
    cols = columns(build_H(P, k, r))
    if dmin == 3:
        if not any(s):
            return "ошибок не обнаружено", None
        for j, c in enumerate(cols):
            if c == s:
                return "одиночная ошибка", j
        return "ошибка обнаружена, но не может быть исправлена", None
    sr, sa = s[:-1], s[-1]
    if not any(sr) and sa == 0:
        return "ошибок не обнаружено", None
    if sa == 1:
        if not any(sr):
            return "одиночная ошибка (в общем паритете)", n - 1
        for j, c in enumerate(cols):
            if c == sr:
                return "одиночная ошибка", j
        return "ошибка обнаружена, но не может быть исправлена (нечётная кратность >= 3)", None
    return "двойная ошибка обнаружена, исправление невозможно", None


def run(xk, dmin, nerr, rng, out):
    k = len(xk)
    r = calc_r(k)
    P = build_P(k, r)
    H = build_H(P, k, r)
    xr = check_bits(xk, P, dmin)
    xn = xk + xr
    n = len(xn)

    pos = sorted(rng.sample(range(n), nerr))
    yn = xn[:]
    for p in pos:
```

```
        yn[p] ^= 1
    yr = yn[k:]
    yr2, s = syndrome(yn, k, P, dmin)
    status, p = locate(s, P, k, dmin)

    en = [0] * n
    if p is not None:
        en[p] = 1
    yfix = [a ^ b for a, b in zip(yn, en)]

    w = out.append
    w(f"d_min = {dmin}")
    w(f"k = {k}, r = {len(xr)}, n = {n}")
    w(f"относительная избыточность r/k = {len(xr) / k:.3f}, скорость k/n = {k / n:.3f}")
    if dmin == 3:
        w("\nПроверочная матрица H (r x n) = [P' | I]:")
        w(matrix_str(H))
    else:
        Hx, Hc = build_H4(H)
        w("\nПроверочная матрица H' (4.8):")
        w(matrix_str(Hx))
        w("\nКаноническая форма (последняя строка = сумма всех строк):")
        w(matrix_str(Hc))
    w(f"\nX_k  (информационное слово)   = {fmt(xk)}")
    w(f"X_r  (избыточные символы)     = {fmt(xr)}")
    w(f"X_n  (кодовое слово)          = {fmt(xn)}")
    w(f"\nГенерируется ошибок: {nerr}; позиции (с 1): {[q + 1 for q in pos]}")
    w(f"Y_n  (принятое слово)         = {fmt(yn)}")
    w(f"Y_r  (принятые избыточные)    = {fmt(yr)}")
    w(f"Y_r' (пересчитанные)          = {fmt(yr2)}")
    w(f"S    (синдром)                = {fmt(s)}")
    w(f"Результат анализа синдрома: {status}" + (f" в позиции {p + 1}" if p is not None else ""))
    w(f"E_n  (вектор ошибки)          = {fmt(en)}")
    w(f"Y_n' (исправленное слово)     = {fmt(yfix)}")
    if yfix == xn:
        w("Итог: Y_n' = X_n, слово восстановлено верно." if nerr else "Итог: ошибок не было, слово
принято верно.")
    elif p is None and any(s):
        w("Итог: ошибка обнаружена, слово не исправлено (требуется повторная передача).")
    elif not any(s):
        w("Итог: синдром нулевой, но слово искажено - ошибка НЕ обнаружена.")
    else:
        w("Итог: слово исправлено НЕВЕРНО (число ошибок превышает корректирующую способность кода).")


def matrices_report():
    out = []
    for k in (4, 6, 8, 9, 10, 15, 16):
        r = calc_r(k)
        P = build_P(k, r)
        H = build_H(P, k, r)
        Hx, Hc = build_H4(H)
        out.append(f"===== k = {k} =====")
        out.append(f"d_min = 3: r = {r}, n = {k + r}\n{matrix_str(H)}")
        out.append(f"d_min    =    4:    r    =    {r    +    1},    n    =    {k    +    r    +
1}\nH':\n{matrix_str(Hx)}\nканоническая:\n{matrix_str(Hc)}\n")
    k = 3
    out.append("===== Код простой чётности, k = 3 (d_min = 2) =====")
    out.append("H = [1 1 1 1]   (X_r = x1 + x2 + x3 mod 2, n = 4)")
    return "\n".join(out)


def main():
    ap   =   argparse.ArgumentParser(description="Код   Хемминга:   кодирование,   ошибки,   синдром,
исправление")
    src = ap.add_mutually_exclusive_group()
    src.add_argument("--text", help="исходное сообщение (строка)")
```

```
    src.add_argument("--file", help="текстовый файл с сообщением")
    ap.add_argument("--dmin", type=int, choices=(3, 4), default=3)
    ap.add_argument("--errors", type=int, choices=(0, 1, 2), default=1)
    ap.add_argument("--seed", type=int, default=None)
    ap.add_argument("--out", default="hamming_log.txt", help="файл-протокол")
    ap.add_argument("--matrices",  action="store_true",  help="вывести  матрицы  для  контрольных
вопросов")
    a = ap.parse_args()

    if a.matrices:
        text = matrices_report()
        print(text)
        with open("hamming_matrices.txt", "w", encoding="utf-8") as f:
            f.write(text)
        return

    if a.file:
        with open(a.file, encoding="utf-8") as f:
            msg = f.read().strip()
    else:
        msg = a.text if a.text else "Hello, мир!"
    xk = bits_from_text(msg)
    if len(xk) < MIN_BITS:
        raise SystemExit(f"Сообщение слишком короткое: {len(xk)} бит, нужно не менее {MIN_BITS}")

    rng = random.Random(a.seed)
    out = [f"Сообщение: {msg!r}"]
    run(xk, a.dmin, a.errors, rng, out)
    text = "\n".join(out)
    print(text)
    with open(a.out, "w", encoding="utf-8") as f:
        f.write(text + "\n")


if __name__ == "__main__":
    main()
```

Iterative.py:

```
import argparse
import csv
import random
from itertools import combinations

K = 32
VARIANT_4 = [
    ((4, 8), (2, 3)),
    ((2, 16), (2, 3)),
    ((8, 2, 2), (2, 3, 4, 5)),
    ((4, 4, 2), (2, 3, 4, 5)),
]
GROUP_NAME = {
    "h": "горизонтальные", "v": "вертикальные", "z": "по оси Z",
    "d1": "диагональные (1)", "d2": "диагональные (2)",
}


def fmt(bits):
    return "".join(map(str, bits))


class IterCode:
    def __init__(self, dims, ngroups):
        self.k1, self.k2 = dims[0], dims[1]
        self.z = dims[2] if len(dims) > 2 else 1
        self.k = self.k1 * self.k2 * self.z
        order = ["h", "v", "z", "d1", "d2"] if self.z > 1 else ["h", "v", "d1"]
```

```
    self.groups = order[:ngroups]
    M = max(self.k1, self.k2)

    def key(g, i, j, p):
        return {"h": (p, i), "v": (p, j), "z": (i, j),
                "d1": (p, (j - i) % M), "d2": (p, (i + j) % M)}[g]

    self.cell = []
    for t in range(self.k):
        p, rem = divmod(t, self.k1 * self.k2)
        self.cell.append((rem // self.k2, rem % self.k2, p))

    self.checks, self.check_group = [], []      линии
    for g in self.groups:
        lines = {}
        for t, (i, j, p) in enumerate(self.cell):
            lines.setdefault(key(g, i, j, p), []).append(t)
        for kk in sorted(lines):
            self.checks.append(lines[kk])
            self.check_group.append(g)
    self.r = len(self.checks)
    self.n = self.k + self.r

def title(self):
    d = "x".join(map(str, (self.k1, self.k2) + ((self.z,) if self.z > 1 else ())))
    return f"{d}, групп паритетов: {len(self.groups)} ({', '.join(self.groups)})"

def encode(self, xk):
    xr = [sum(xk[t] for t in idx) % 2 for idx in self.checks]
    return xk + xr

def syndrome(self, y):
    return [(sum(y[t] for t in idx) + y[self.k + c]) % 2 for c, idx in enumerate(self.checks)]

def _tables(self):
    if hasattr(self, "_col"):
        return
    col = [0] * self.n
    for c, idx in enumerate(self.checks):
        for t in idx:
            col[t] |= 1 << c
        col[self.k + c] |= 1 << c
    self._col = col
    self._single, self._pairs = {}, {}
    for a in range(self.n):
        self._single.setdefault(col[a], []).append(a)
        for b in range(a + 1, self.n):
            self._pairs.setdefault(col[a] ^ col[b], []).append((a, b))

def decode(self, y):
    self._tables()
    col = self._col
    y = y[:]
    s = 0
    for b in range(self.n):
        if y[b]:
            s ^= col[b]
    if s == 0:
        return y, 0, True
    found = self._single.get(s, [])
    if found:
        cands = [(a,) for a in found]
    else:
        cands = self._pairs.get(s, [])
        if not cands:
            cs = set()
            for b in range(self.n):
```

```
                    for (a, c) in self._pairs.get(s ^ col[b], []):
                        if b != a and b != c:
                            cs.add(tuple(sorted((a, b, c))))
                cands = list(cs)
        if len(cands) != 1:
            return y, 0, False
        for q in cands[0]:
            y[q] ^= 1
        return y, len(cands[0]), True

    def dmin(self, max_w=7):
        col = [0] * self.n
        for c, idx in enumerate(self.checks):
            for t in idx:
                col[t] |= 1 << c
            col[self.k + c] |= 1 << c
        for w in range(1, max_w + 1):
            a = w // 2
            if w == 1:
                if any(c == 0 for c in col):
                    return 1
                continue
            seen = set()
            if w % 2 == 0:
                for sub in combinations(range(self.n), a):
                    x = 0
                    for b in sub:
                        x ^= col[b]
                    if x in seen:
                        return w
                    seen.add(x)
            else:
                for sub in combinations(range(self.n), a):
                    x = 0
                    for b in sub:
                        x ^= col[b]
                    seen.add(x)
                for sub in combinations(range(self.n), a + 1):
                    x = 0
                    for b in sub:
                        x ^= col[b]
                    if x in seen:
                        return w
        return None


def demo(code, xk, nerr, rng, out):
    w = out.append
    w("=" * 70)
    w(f"Код: {code.title()}")
    w(f"k = {code.k}, r = {code.r}, n = {code.n}, относительная избыточность r/k = {code.r /
code.k:.3f}, "
      f"скорость k/n = {code.k / code.n:.3f}")
    w(f"X_k = {fmt(xk)}")
    for p in range(code.z):
        w(f"\nИнформационная матрица{' (плоскость %d)' % (p + 1) if code.z > 1 else ''}
{code.k1}x{code.k2}:")
        for i in range(code.k1):
            base = p * code.k1 * code.k2 + i * code.k2
            w("  " + " ".join(map(str, xk[base:base + code.k2])))
    xn = code.encode(xk)
    w("\nИзбыточные символы по группам:")
    pos = code.k
    for g in code.groups:
        cnt = code.check_group.count(g)
        w(f"  {GROUP_NAME[g]:<18}({cnt:>2} бит): {fmt(xn[pos:pos + cnt])}")
        pos += cnt
```

```
    w(f"\nX_n = {fmt(xn)}")

    err = sorted(rng.sample(range(code.n), nerr))
    yn = xn[:]
    for q in err:
        yn[q] ^= 1
    s = code.syndrome(yn)
    w(f"\nОшибок: {nerr}, позиции (с 1): {[q + 1 for q in err]}")
    w(f"Y_n = {fmt(yn)}")
    w(f"Синдром (нарушены проверки №): {[c + 1 for c, v in enumerate(s) if v] or 'нет'}")
    yfix, flips, ok = code.decode(yn)
    changed = [q + 1 for q in range(code.n) if yfix[q] != yn[q]]
    w(f"Исправлены позиции: {changed or 'нет'}")
    w(f"Y_n' = {fmt(yfix)}")
    if yfix == xn:
        w("Итог: Y_n' = X_n, слово восстановлено верно." if nerr else "Итог: ошибок не было.")
    elif not ok:
        w("Итог: ошибка обнаружена, исправить однозначно нельзя.")
    else:
        w("Итог: слово исправлено неверно / ошибка не обнаружена.")


def experiment(code, n1, imax, rng):
    rows = []
    for i in range(1, imax + 1):
        n_det = n2 = n3 = 0
        for _ in range(n1):
            xk = [rng.randint(0, 1) for _ in range(code.k)]
            xn = code.encode(xk)
            yn = xn[:]
            for q in rng.sample(range(code.n), i):
                yn[q] ^= 1
            if any(code.syndrome(yn)):
                n_det += 1
            yfix, flips, ok = code.decode(yn)
            if ok and flips == i:
                n2 += 1
            if yfix == xn:
                n3 += 1
        rows.append((i, n1, n_det, n2, n3))
    return rows


def main():
    ap = argparse.ArgumentParser(description="Итеративные коды, вариант 4 (k = 32)")
    ap.add_argument("--demo", action="store_true")
    ap.add_argument("--experiment", action="store_true")
    ap.add_argument("--word", help="информационное слово из 32 бит (по умолчанию случайное)")
    ap.add_argument("--errors", type=int, default=1, help="кратность ошибки в демонстрации")
    ap.add_argument("--n1", type=int, default=3000, help="число слов N1 на каждую кратность")
    ap.add_argument("--imax", type=int, default=6, help="максимальная кратность ошибки")
    ap.add_argument("--seed", type=int, default=4)
    a = ap.parse_args()
    rng = random.Random(a.seed)
    if not (a.demo or a.experiment):
        a.demo = a.experiment = True

    if a.demo:
        out = []
        for dims, gs in VARIANT_4:
            for g in gs:
                xk = [int(c) for c in a.word] if a.word else [rng.randint(0, 1) for _ in range(K)]
                if len(xk) != K:
                    raise SystemExit("--word должно содержать ровно 32 бита")
                demo(IterCode(dims, g), xk, a.errors, rng, out)
        text = "\n".join(out)
        print(text)
```

```
        with open("iterative_demo.txt", "w", encoding="utf-8") as f:
            f.write(text + "\n")

    if a.experiment:
        lines = [f"Эксперимент: N1 = {a.n1} слов на каждую кратность i, seed = {a.seed}",
                 "N2/N1 - кратность ошибки определена верно, N3/N1 - все ошибки исправлены, Nобн/N1
- ошибка обнаружена", ""]
        csv_rows = []
        for dims, gs in VARIANT_4:
            for g in gs:
                code = IterCode(dims, g)
                d = code.dmin()
                lines.append(f"--- {code.title()} | k={code.k}, r={code.r}, n={code.n}, "
                             f"r/k={code.r / code.k:.3f}, k/n={code.k / code.n:.3f}, d_min={d if d
else '>7'}")
                lines.append(f"{'i':>3} {'N1':>7} {'Nобн/N1':>9} {'N2/N1':>8} {'N3/N1':>8}")
                for i, n1, nd, n2, n3 in experiment(code, a.n1, a.imax, rng):
                    lines.append(f"{i:>3} {n1:>7} {nd / n1:>9.4f} {n2 / n1:>8.4f} {n3 / n1:>8.4f}")
                    csv_rows.append([code.title(), code.r, f"{code.r / code.k:.3f}", d, i, n1,
                                     f"{nd / n1:.4f}", f"{n2 / n1:.4f}", f"{n3 / n1:.4f}"])
                lines.append("")
        text = "\n".join(lines)
        print(text)
        with open("iterative_experiment.txt", "w", encoding="utf-8") as f:
            f.write(text + "\n")
        with open("iterative_experiment.csv", "w", encoding="utf-8", newline="") as f:
            wr = csv.writer(f)
            wr.writerow(["код", "r", "r/k", "d_min", "i", "N1", "Nобн/N1", "N2/N1", "N3/N1"])
            wr.writerows(csv_rows)


if __name__ == "__main__":
    main()
```

Хемминг (одна ошибка, исправлена):

```
PS D:\politex\ЛАБЫ\7 семестр\КиСБ\lab_2> python hamming.py --text "Hi" --dmin 3 --error
s 1 --seed 1
Сообщение: 'Hi'
d_min = 3
k = 16, r = 5, n = 21
относительная избыточность r/k = 0.312, скорость k/n = 0.762

Проверочная матрица H (r x n) = [P' | I]:
0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0
0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 1 0 0 0 0
0 1 1 1 0 0 0 1 1 1 1 0 0 0 1 1 0 0 1 0 0 0
1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 0 0 0 0 1 0 0
1 1 0 1 1 0 1 0 1 0 1 1 0 1 0 1 0 0 0 0 1 0

X_k  (информационное слово)   = 0100100001101001
X_r  (избыточные символы)     = 01010
X_n  (кодовое слово)          = 010010000110100101010

Генерируется ошибок: 1; позиции (с 1): [5]
Y_n  (принятое слово)         = 010000000110100101010
Y_r  (принятые избыточные)    = 01010
Y_r' (пересчитанные)          = 00011
S    (синдром)                = 01001
Результат анализа синдрома: одиночная ошибка в позиции 5
E_n  (вектор ошибки)          = 000010000000000000000
Y_n' (исправленное слово)     = 010010000110100101010
Итог: Y_n' = X_n, слово восстановлено верно.
```

Итеративный код (эксперимент с таблицей результатов):

```
PS D:\politex\ЛАБЫ\7 семестр\КиСБ\lab_2> python iterative.py --experiment --n1 300
Эксперимент: N1 = 300 слов на каждую кратность i, seed = 4
N2/N1 - кратность ошибки определена верно, N3/N1 - все ошибки исправлены, Nобн/N1 - ошибка обнаружена

--- 4x8, групп паритетов: 2 (h, v) | k=32, r=12, n=44, r/k=0.375, k/n=0.727, d_min=3
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.0000   0.0000
  3     300    0.9967   0.0000   0.0000
  4     300    1.0000   0.0000   0.0000
  5     300    0.9967   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 4x8, групп паритетов: 3 (h, v, d1) | k=32, r=20, n=52, r/k=0.625, k/n=0.615, d_min=4
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.8733   0.8733
  3     300    1.0000   0.6033   0.6033
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 2x16, групп паритетов: 2 (h, v) | k=32, r=18, n=50, r/k=0.562, k/n=0.640, d_min=3
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.0000   0.0000
  3     300    0.9967   0.0000   0.0000
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 2x16, групп паритетов: 3 (h, v, d1) | k=32, r=34, n=66, r/k=1.062, k/n=0.485, d_min=4
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.9167   0.9167
  3     300    1.0000   0.8067   0.8067
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000
```

```
--- 8x2x2, групп паритетов: 2 (h, v) | k=32, r=20, n=52, r/k=0.625, k/n=0.615, d_min=3
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.5333   0.5333
  3     300    1.0000   0.0000   0.0000
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 8x2x2, групп паритетов: 3 (h, v, z) | k=32, r=36, n=68, r/k=1.125, k/n=0.471, d_min=4
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.9400   0.9400
  3     300    1.0000   0.7733   0.7733
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 8x2x2, групп паритетов: 4 (h, v, z, d1) | k=32, r=52, n=84, r/k=1.625, k/n=0.381, d_min=5
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   1.0000   1.0000
  3     300    1.0000   0.9967   0.9967
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 8x2x2, групп паритетов: 5 (h, v, z, d1, d2) | k=32, r=68, n=100, r/k=2.125, k/n=0.320, d_min=6
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   1.0000   1.0000
  3     300    1.0000   1.0000   1.0000
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000
```

```
--- 4x4x2, групп паритетов: 2 (h, v) | k=32, r=16, n=48, r/k=0.500, k/n=0.667, d_min=3
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.5133   0.5133
  3     300    1.0000   0.0000   0.0000
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 4x4x2, групп паритетов: 3 (h, v, z) | k=32, r=32, n=64, r/k=1.000, k/n=0.500, d_min=4
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   0.9000   0.9000
  3     300    1.0000   0.7633   0.7633
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 4x4x2, групп паритетов: 4 (h, v, z, d1) | k=32, r=40, n=72, r/k=1.250, k/n=0.444, d_min=5
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   1.0000   1.0000
  3     300    1.0000   0.9933   0.9933
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000

--- 4x4x2, групп паритетов: 5 (h, v, z, d1, d2) | k=32, r=48, n=80, r/k=1.500, k/n=0.400, d_min=6
  i      N1   Nобн/N1    N2/N1    N3/N1
  1     300    1.0000   1.0000   1.0000
  2     300    1.0000   1.0000   1.0000
  3     300    1.0000   0.9833   0.9833
  4     300    1.0000   0.0000   0.0000
  5     300    1.0000   0.0000   0.0000
  6     300    1.0000   0.0000   0.0000
```

Вывод: приобрели практические навыки кодирования/декодирования двоичных данных при использовании кода Хемминга и итеративных кодов.